

INTRODUZIONE ALLA PROGRAMMAZIONE SU ARM – Alessandro Simonetta

▪ Capitolo 1

L'evoluzione tecnologica negli ultimi decenni ha permesso di concentrare funzioni di calcolo complesse in minore spazio. Questa riduzione di spazio, insieme all'abbassamento dei consumi energetici e alla realizzazione di batterie sempre più piccole e capienti, ha permesso di progettare dispositivi mobili con straordinarie capacità computazionali. Questi dispositivi digitali hanno sostituito i dispositivi analogici. Infatti è risultato necessario tradurre le grandezze analogiche in grandezze digitali (DAC: Digital to Analog), e viceversa (ADC: Analog to Digital).

Questo processo avviene attraverso 3 fasi:

- Campionamento: Si prendono misure del segnale a intervalli regolari nel tempo, convertendo così il segnale analogico in una sequenza di valori discreti.
- Quantizzazione: Si raggruppano questi valori in intervalli chiamati livelli di quantizzazione, rendendo i valori discreti. Questo passaggio è non lineare e gli intervalli possono essere più o meno precisi.
- Codifica: Si associa ad ogni valore quantizzato un codice binario univoco. La scelta del numero di valori e del numero di campioni influisce sulla precisione e sulla dimensione del dato digitale.

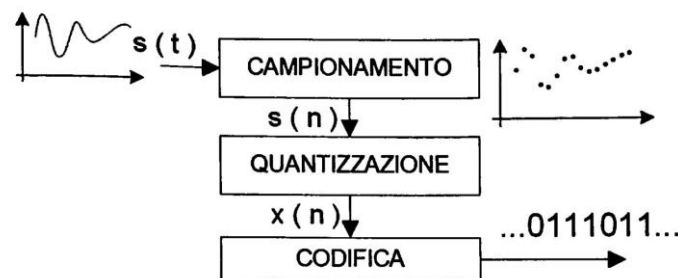


Figura 1.3: Conversione A/D.

Abbiamo varie tipologie di **SISTEMI DI NUMERAZIONE**, ovvero:

Sistema Decimale → è il sistema di numerazione posizionale attuale

$$(\text{numero})_{10} = \sum_{k=0}^{n-1} x_k \cdot B^k$$

Sistema Binario → è un sistema posizionale in base 2 che utilizza il seguente alfabeto $\{0, 1\}$. In questo sistema una variabile può assumere due soli valori, infatti viene detta bit (Binary digIT). Quindi, i numeri nel sistema binario sono effettivamente formati da bit, che possono essere 0 o 1.

ES: $(111110)_2$

Possiamo trasformare un numero da un sistema Binario ad un sistema Decimale e viceversa.

Da Binario a Decimale: applichiamo la seguente regola

$$\sum_{0}^k x_k \cdot 2^k$$

Dove k = posizione della cifra del numero binario e x = bit del numero binario

ES: Avendo il seguente numero binario $(111110)_2$ possiamo notare che è formato da 6 bit (da 0 a 5). Quindi avremo k che va da 0 a 5. Infatti andrò ad applicare la regola su scrivendo:

$$1 \cdot 2^5 + 1 \cdot 2^4 + 1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0 = 32 + 16 + 8 + 4 + 2 + 0 = (62)_{10}$$

Da Decimale a Binario: applichiamo una divisione in colonna diviso 2, nel seguente modo:

- Scrivi nella colonna di sinistra il numero decimale da convertire
- Dividi il numero diviso 2
- Il quoziente della divisione lo scriveremo nella colonna di sinistra
- Il resto della divisione lo scriveremo nella colonna di destra, scrivendo:
 - 0 se è pari
 - 1 se è dispari

Ripeti queste operazioni fin quando nei valori decimali si arriva a zero, allora il processo termina.

ES:

62	0
31	1
15	1
7	1
3	1
1	1

Quindi $(62)_{10} = (111110)_2$

Sistema Ottale → ha le stesse proprietà del sistema binario, solo che una cifra ottale corrisponde a 3 cifre binarie (2^3)

ES:

4	2	1	4	2	1	4	2	1
0	0	1	1	1	1	1	1	0
1			7				6	

In questo caso $(1111110)_2 = (0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0) | (1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) | (1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) = 1 | 7 | 6 = (176)_8$

Sistema Esadecimale → ha le stesse proprietà del sistema binario, solo che una cifra ottale corrisponde a 4 cifre binarie (2^4)

ES:

8	4	2	1	8	4	2	1
0	0	1	1	1	1	1	0
3				E			

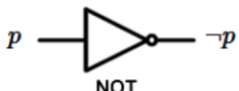
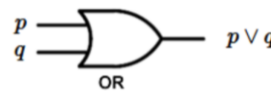
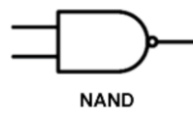
In questo caso $(1111110)_2 = (0 \cdot 2^3 + 0 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0) | (1 \cdot 2^3 + 1 \cdot 2^2 + 1 \cdot 2^1 + 0 \cdot 2^0) = 3 | 14 = (3E)_8$

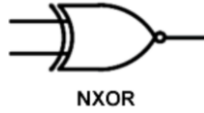
N.B: nel sistema esadecimale i numeri da 0 a 9 corrispondono con il sistema decimale, dal 10 in poi corrispondono alle lettere dell'alfabeto (10 = A, 11 = B, ecc...)

decimale	esadecimale	ottale	binario
0	0	0	0000
1	1	1	0001
2	2	2	0010
3	3	3	0011
4	4	4	0100
5	5	5	0101
6	6	6	0110
7	7	7	0111
8	8	10	1000
9	9	11	1001
10	A	12	1010
11	B	13	1011
12	C	14	1100
13	D	15	1101
14	E	16	1110
15	F	17	1111

Tabella 1.2: Tabella di corrispondenza dei numeri nelle differenti basi.

L'origine storica della logica la possiamo datare nella prima metà del diciannovesimo secolo grazie a George Boole. Quest'ultimo creò l'**ALGEBRA DI BOOLE**, cioè un'algebra astratta che utilizza solo i valori di verità 1 = True e 0 = False. Poiché utilizza 0 e 1 è chiamata pure algebra binaria. Le operazioni booleane sono realizzate dalle porte logiche, ovvero circuiti elettronici digitali. Tutto ciò ci permetterà di costruire circuiti più complessi fino a descrivere l'architettura di un calcolatore.

Identità	$x = A$		<table><tr><th>A</th><th>x</th></tr><tr><td>1</td><td>1</td></tr><tr><td>0</td><td>0</td></tr></table>	A	x	1	1	0	0									
A	x																	
1	1																	
0	0																	
Negazione (NOT)	$x = \bar{A}$		<table><tr><th>p</th><th>$\neg p$</th></tr><tr><td>0</td><td>1</td></tr><tr><td>1</td><td>0</td></tr></table>	p	$\neg p$	0	1	1	0									
p	$\neg p$																	
0	1																	
1	0																	
Somma Logica (OR)	$x = A + B$		<table><tr><th>p</th><th>q</th><th>$p \vee q$</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	p	q	$p \vee q$	0	0	0	0	1	1	1	0	1	1	1	1
p	q	$p \vee q$																
0	0	0																
0	1	1																
1	0	1																
1	1	1																
Somma Logica Invertita (NOR)	$x = \overline{A + B}$		<table><tr><th>p</th><th>q</th><th>$\overline{p \vee q}$</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	p	q	$\overline{p \vee q}$	0	0	1	0	1	0	1	0	0	1	1	0
p	q	$\overline{p \vee q}$																
0	0	1																
0	1	0																
1	0	0																
1	1	0																
Prodotto Logico (AND)	$x = A \cdot B$		<table><tr><th>p</th><th>q</th><th>$p \wedge q$</th></tr><tr><td>0</td><td>0</td><td>0</td></tr><tr><td>0</td><td>1</td><td>0</td></tr><tr><td>1</td><td>0</td><td>0</td></tr><tr><td>1</td><td>1</td><td>1</td></tr></table>	p	q	$p \wedge q$	0	0	0	0	1	0	1	0	0	1	1	1
p	q	$p \wedge q$																
0	0	0																
0	1	0																
1	0	0																
1	1	1																
Prodotto Logico Invertito (NAND)	$x = \overline{A \cdot B}$		<table><tr><th>p</th><th>q</th><th>$\overline{p \wedge q}$</th></tr><tr><td>0</td><td>0</td><td>1</td></tr><tr><td>0</td><td>1</td><td>1</td></tr><tr><td>1</td><td>0</td><td>1</td></tr><tr><td>1</td><td>1</td><td>0</td></tr></table>	p	q	$\overline{p \wedge q}$	0	0	1	0	1	1	1	0	1	1	1	0
p	q	$\overline{p \wedge q}$																
0	0	1																
0	1	1																
1	0	1																
1	1	0																

OR Esclusivo (XOR)	$x = A \oplus B$	 XOR	<table> <tr> <th>p</th> <th>q</th> <th>$p \dot{\vee} q$</th> </tr> <tr><td>0</td><td>0</td><td>0</td></tr> <tr><td>0</td><td>1</td><td>1</td></tr> <tr><td>1</td><td>0</td><td>1</td></tr> <tr><td>1</td><td>1</td><td>0</td></tr> </table>	p	q	$p \dot{\vee} q$	0	0	0	0	1	1	1	0	1	1	1	0
p	q	$p \dot{\vee} q$																
0	0	0																
0	1	1																
1	0	1																
1	1	0																
NOR Esclusivo (XNOR)	$x = \overline{A \oplus B}$	 NXOR	<table> <tr> <th>p</th> <th>q</th> <th>$\overline{p \dot{\vee} q}$</th> </tr> <tr><td>0</td><td>0</td><td>1</td></tr> <tr><td>0</td><td>1</td><td>0</td></tr> <tr><td>1</td><td>0</td><td>0</td></tr> <tr><td>1</td><td>1</td><td>1</td></tr> </table>	p	q	$\overline{p \dot{\vee} q}$	0	0	1	0	1	0	1	0	0	1	1	1
p	q	$\overline{p \dot{\vee} q}$																
0	0	1																
0	1	0																
1	0	0																
1	1	1																

• **Proprietà per le semplificazioni:**

$$\overline{A + B} = \bar{A} \cdot \bar{B} \quad oppure \quad \overline{A \cdot B} = \bar{A} + \bar{B} \quad (\text{De Morgan})$$

$$A + B = B + A \quad oppure \quad A \cdot B = B \cdot A \quad (\text{proprietà commutativa})$$

$$A \cdot (B + C) = A \cdot B + A \cdot C \quad o \quad A + (B \cdot C) = (A + B)(A + C) \quad (\text{proprietà distributiva})$$

$$(AB)C = A(BC) \quad o \quad (A + B) + C = A + (B + C) \quad (\text{proprietà associativa})$$

$$A + (\bar{A} \cdot B) = A + B \quad o \quad A + (A \cdot B) = A \quad (\text{proprietà dell'assorbimento})$$

$$A + 0 = A$$

$$A + 1 = 1$$

$$A + A = A$$

$$A + \bar{A} = 1$$

$$A \cdot 0 = 0$$

$$A \cdot 1 = A$$

$$A \cdot A = A$$

$$A \cdot \bar{A} = 0$$

DEFINIZIONE DI NUOVE FUNZIONI

L'ambito di applicazione dell'algebra di Boole è talmente vasto che ci permette di studiare varie problematiche. Per esempio supponiamo una progettazione di un sistema di allarme per un locale, che può essere attivato tramite un comando cellulare e deve suonare se almeno una delle due entrate (porta o finestra) è aperta e non è disattivato. Utilizzando l'algebra di Boole, possiamo identificare le variabili di ingresso:

- Comando di abilitazione (E)

- Attivazione dei due sensori, per mezzo del cellulare, installati rispettivamente sulla porta (P)
- e sulla finestra (W)

Attraverso tavole di verità, vengono esplorate tutte le combinazioni di stato e si individua la relazione logica tra gli ingressi e l'uscita.



Figura 1.10: Schema di contesto del problema.

E $\begin{cases} 0 & \text{allarme non inserito} \\ 1 & \text{allarme attivo} \end{cases}$

P $\begin{cases} 0 & \text{porta aperta} \\ 1 & \text{porta chiusa} \end{cases}$

W $\begin{cases} 0 & \text{finestra aperta} \\ 1 & \text{finestra chiusa} \end{cases}$

A $\begin{cases} 0 & \text{allarme spento} \\ 1 & \text{allarme suona} \end{cases}$

E	P	W	A
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	1
1	1	0	1
1	1	1	0

Per esprimere l'espressione logica finale, usiamo la **forma canonica disgiuntiva**, detta anche DNF (considera nella tavola di verità in output gli 1 e scrivi come somma di prodotti) e **forma canonica congiuntiva**, detta anche CNF (considera nella tavola di verità gli 0 e scrivi come prodotti di somme)

Quindi in questo caso avremo:

- DNF:

$$A = E \cdot \bar{P} \cdot \bar{W} + E \cdot \bar{P} \cdot W + E \cdot P \cdot \bar{W}$$

- CNF:

$$A = (E + P + W) \cdot (E + P + \bar{W}) \cdot (E + \bar{P} + W) \cdot (E + \bar{P} + \bar{W}) \cdot (\bar{E} + \bar{P} + \bar{W})$$

EQUIVALENZE E OPERATORI UNIVERSALI

NOT	 $\bar{A}$
NAND	 $\overline{A \cdot A} = \bar{A}$
NOR	 $\overline{A + A} = \bar{A}$

Tabella 1.6: Porte logiche equivalenti alla porta invertente

AND	 $A \cdot B$
NAND	 $\overline{A \cdot B} = A \cdot B$
NOR	 $\overline{A + B} = A \cdot B$

Tabella 1.7: Porte logiche equivalenti alla porta AND

OR	 $A + B$
NAND	 $\overline{A \cdot B} = A + B$
NOR	 $\overline{A + B} = A \cdot B$

Tabella 1.8: Porte logiche equivalenti alla porta AND

LE TRASFORMAZIONI NEL DOMINIO DI BOOLE



Figura 1.11: Le trasformazioni nel dominio di Boole.

La **MAPPA DI KARNAUGH** o **MAPPE K** è un metodo di semplificazione algebrica che richiede meno passaggi. Questo metodo consiste nel creare una tabella con 2^r righe e 2^c colonne, dove $r + c = n^\circ$ variabili logiche. Questa tabella è la rappresentazione piana di una dimensione toroidale realizzata avvolgendo le estremità. Per scrivere il risultato raggruppa i termini adiacenti in gruppi di 2^n (cioè potenze di 2) e scrivi l'espressione come somme di prodotti.

$A \backslash B$	0	1
0		
1		

$A \backslash BC$	00	01	11	10
0				
1				

$AB \backslash CD$	00	01	11	10
00				
01				
11				
10				

Figura 1.12: Esempi di mappe K con 2, 3 e 4 variabili in ingresso